

MOBILE APPLICATION AND DEVELOPMENT – WEEK 9

Data Storage & Persistence – Shared Preferences

Key/value storage · Reading and writing preferences · A real login-flag example

Ekramul Haque Amin

Lecturer, Dept. of AI & Data Science

TODAY'S SESSION – THEORY 2

Agenda

01 Why Shared Preferences exists, and where it fits next to SQLite/Room

02 Getting a handle, reading, and writing with the Editor

03 A real example – remembering a logged-in user

04 Common mistakes, and when to reach for something else instead

FOLLOWING ON FROM LAST CLASS

Not Every Value Needs a Database

Last class we built SQLite tables and a Room database for structured, queryable records. But a single "is dark mode on?" flag doesn't need tables, a schema, or SQL — it needs one key and one value.

SQLite / Room

Many rows, structured, queryable — a list of contacts or expenses

Shared Preferences

A handful of flat values — a flag, a name, a setting

Today we cover the second one — the simplest persistence option in Android, and one of the most used.

PART 1 – SHARED PREFERENCES

What Is Shared Preferences?

One of the most useful storage options in Android — it stores small amounts of primitive data (strings, ints, floats, booleans) as key/value pairs, saved in an XML file inside the app's own storage.

THINK OF IT LIKE

A dictionary — every key ("username") has one associated value, read and written by that name.

Typical use: a "logged in" flag, a saved username, a theme preference — never a whole list of records.

WHERE IT ACTUALLY LIVES

Just an XML File on Disk

Every named preferences file is saved at `/data/data/<package>/shared_prefs/<name>.xml`. There's no magic — it's a plain XML file the framework reads and rewrites for you.

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <boolean name="flag" value="true" />
  <string name="username">habib123</string>
</map>
```

Viewable on a rooted device or emulator — handy for debugging when a value isn't showing up the way you expect.

WHERE YOU'LL ACTUALLY USE THIS

Common Use Cases

Login state

"Is this user logged in?" — skip the login screen on relaunch

Onboarding flag

"Has this user seen the tutorial?" — show it only once

App settings

Theme, notification toggle, preferred units

Small cached values

The last search query, a scroll position, a draft's timestamp

GETTING A HANDLE ON IT

getSharedPreferences()

```
SharedPreferences preferences =  
    getSharedPreferences("login", MODE_PRIVATE);
```

MODE_PRIVATE

Private to this app — the recommended,
secure default

MODE_PUBLIC (deprecated)

Accessible to other apps — no longer recommended

MODE_APPEND

Used when appending while reading the file

TWO WAYS TO ASK FOR ONE

getPreferences() vs. getSharedPreferences()

getPreferences(MODE_PRIVATE)

An Activity-only shortcut — one file, automatically named after that Activity's class. No name to remember.

getSharedPreferences("login", MODE_PRIVATE)

You choose the file name — so any Activity or Fragment that uses the same name reads the same data.

Our login example needs LoginActivity and HomePage to share one flag — that's why it names the file explicitly instead of using getPreferences().

READING AND WRITING

The Editor & Its Methods

edit()

Creates an Editor to make and commit changes

contains(key)

Checks whether a preference exists

getBoolean / getString / getInt...

Read a typed value, with a default

getAll()

Retrieves every stored value at once

register/unregisterOnSharedPreferenceChangeListener()

Runs a callback on the main thread when a value changes

commit() vs. apply()

commit() saves synchronously; apply() saves in the background

ONLY FIVE PRIMITIVE SHAPES

Every put/get Pair

Type	Write	Read
String	putString(key, val)	getString(key, def)
int	putInt(key, val)	getInt(key, def)
boolean	putBoolean(key, val)	getBoolean(key, def)
float	putFloat(key, val)	getFloat(key, def)
long	putLong(key, val)	getLong(key, def)
Set<String>	putStringSet(key, val)	getStringSet(key, def)

That's it — no object graphs, no nested structures. A "def" default is always required so a first-run read never crashes.

A CHOICE THAT MATTERS

apply() vs. commit()

apply()

Writes to memory immediately, saves to disk asynchronously. Returns void — no result to check.

Use this almost always.

commit()

Writes to disk synchronously and blocks the calling thread. Returns a boolean success flag.

Use only when you must confirm the write finished before the next line runs.

Calling `commit()` on the main thread can cause a visible stutter — it's a common source of jank in apps that get this wrong.

TAKING VALUES BACK OUT

remove() and clear()

```
SharedPreferences.Editor editor = preferences.edit();  
editor.remove("username"); // deletes just this one key  
editor.clear();           // wipes every key in this file  
editor.apply();
```

A logout button typically calls `remove("flag")` or `clear()` rather than `putBoolean("flag", false)` — it depends whether other saved values (like a remembered username) should also disappear.

REACTING TO A CHANGE

OnSharedPreferenceChangeListener

```
preferences.registerOnSharedPreferenceChangeListener(  
    (prefs, key) → {  
        if (key.equals("dark_mode")) {  
            applyTheme(prefs.getBoolean(key, false));  
        }  
    }  
);
```

Useful when a settings screen changes a value another Activity needs to react to. Always unregister the listener in `onDestroy()` to avoid a memory leak.

ONE FILE PER APP, INSTEAD OF PER FEATURE

The Default Preferences File

`getSharedPreferences("login", ...)` names one specific file. Most apps instead use a single shared, app-wide file — the same one a Settings screen built with the Preference library reads from.

```
SharedPreferences preferences =  
    PreferenceManager.getDefaultSharedPreferences(context);
```

A REAL, WORKING EXAMPLE

Remembering a Logged-In User

Three activities. A 4-second splash screen checks one boolean flag and decides where to send the user next.



```
Boolean check = preferences.getBoolean("flag", false);
startActivity(check ? new Intent(this, HomePage.class) : new Intent(this, LoginActivity.class));
```

WRITING THE FLAG

LoginActivity.java

```
loginbtn.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View view) {  
        SharedPreferences preferences =  
            getSharedPreferences("login", MODE_PRIVATE);  
        SharedPreferences.Editor editor = preferences.edit();  
        editor.putBoolean("flag", true);  
        editor.apply();  
        startActivity(new Intent(LoginActivity.this, HomePage.class));  
    }  
});
```

CLEARING THE FLAG

HomePage.java

```
logoutbtn.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View view) {  
        SharedPreferences preferences =  
            getSharedPreferences("login", MODE_PRIVATE);  
        SharedPreferences.Editor editor = preferences.edit();  
        editor.putBoolean("flag", false);  
        editor.apply();  
        startActivity(new Intent(HomePage.this, MainActivity.class));  
    }  
});
```

Same pattern every time: get preferences → edit() → put... → apply().

A SECOND REAL EXAMPLE

A Dark Mode Switch

```
SharedPreferences prefs = PreferenceManager.getDefaultSharedPreferences(this);
boolean isDark = prefs.getBoolean("dark_mode", false);
darkSwitch.setChecked(isDark);

darkSwitch.setOnCheckedChangeListener((btn, checked) → {
    prefs.edit().putBoolean("dark_mode", checked).apply();
    AppCompatActivity.setDefaultNightMode(checked
        ? AppCompatActivity.MODE_NIGHT_YES
        : AppCompatActivity.MODE_NIGHT_NO);
});
```

Read the saved value once on startup to restore the setting, and write it once on every change — the same read/write shape as the login flag, just a different key.

LEARN FROM THESE BEFORE YOU HIT THEM

Common Mistakes

Storing sensitive data

The XML file is plain text on disk — never put a password or auth token in it directly

Storing structured or large data

A list of 500 contacts belongs in Room, not serialized into one giant preference string

Forgetting `apply()` or `commit()`

Calling `edit().putString(...)` alone changes nothing — the write never happens without one of them

Mixing up which file you're reading

`getSharedPreferences("login", ...)` and the default preferences file are NOT the same file

WHEN YOU ACTUALLY NEED TO STORE SOMETHING SENSITIVE

EncryptedSharedPreferences

We said never put a token or password in plain SharedPreferences. If you truly need to store one locally, Jetpack Security provides a drop-in replacement that encrypts both keys and values on disk.

```
MasterKey masterKey = new MasterKey.Builder(context)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM).build();

SharedPreferences prefs = EncryptedSharedPreferences.create(
    context, "secret_prefs", masterKey,
    EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
    EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM);
```

Same put/get API as before — the encryption is invisible to the rest of your code.

WHILE YOU'RE BUILDING THE LAB

Resetting State While Testing

A saved flag can make it look like your app is broken when it's actually just remembering yesterday's test run. Know how to reset it.

Settings → Apps → Clear Data

Wipes all app storage, including every preferences file

adb shell pm clear <package>

Same reset, from the terminal — faster during repeated test runs

A temporary "reset" button

Wire it to `editor.clear().apply()` while developing, then remove it

GOOD TO KNOW ABOUT, NOT REQUIRED FOR THIS COURSE

Jetpack DataStore

Google's newer replacement for SharedPreferences. Same job — small key/value or typed data — but built on Kotlin coroutines and Flow, with no risk of blocking the UI thread.

SharedPreferences

Synchronous API, no built-in error handling, what we use in this course

DataStore

Asynchronous, transactional, and type-safe with Proto DataStore

Everything you learn today about when to use key/value storage still applies — DataStore is a newer implementation of the same idea.

PART 3 – RECAP

Choosing Between Them

A handful of flat settings or flags? → **Shared Preferences**

A list of structured records you'll query, sort, or filter? → **Room (built on SQLite, from last class)**

Need it fast, with almost no boilerplate? → **Shared Preferences wins on simplicity**

TRY IT NOW – 10 MINUTES

In-Class Exercise

Open the project from today's demo and extend it:

- 1 On LoginActivity, save the entered username with `putString()`, alongside the flag**
- 2 On HomePage, read it back and show "Welcome, <username>" in a TextView**
- 3 Make logout call `remove("username")` as well as `putBoolean("flag", false)`**

BEFORE YOU WRITE A LINE OF PREFERENCES CODE

A Quick Checklist

- 1** Is this a flat value, or a list of records? (records belong in Room)
- 2** Which preferences file — a named one, or the app-wide default?
- 3** Did you call `apply()` (or `commit()`) after `edit()`?
- 4** Is it sensitive? If yes, this is the wrong place for it.

THIS WEEK'S LAB

Putting Both Classes Together

Build the SharedPreferences demo

Recreate the splash → login → home flag flow from today

Build a To-Do List App using Room

Entity, DAO, Database and an Activity, from last class

Decide, don't default

For every new value your app needs to save, ask which of today's two tools actually fits

Next week (Week 10): Firebase Realtime Database and Authentication.

Questions?

Shared Preferences · Editor · apply() vs. commit() · Listeners

Ekramul Haque Amin

Lecturer, Dept. of AI & Data Science